

Documentation for a Sparse Algebra of Gaussian Rational Radical Numbers with `qi_algebra.py`

1 Purpose and scope

This document describes the mathematical model and implementation strategy for the `QINumber` class whose elements are finite sums of the form

$$A = \sum_{j=1}^{N_{\text{terms}}} \frac{p_j + iq_j}{d_j} \sqrt{r_j}, \quad (1)$$

where

$$p_j, q_j, d_j, r_j \in \mathbb{Z}, \quad d_j > 0,$$

and where the sum is finite. The coefficient

$$\frac{p_j + iq_j}{d_j}$$

is a Gaussian rational number, i.e. an element of $\mathbb{Q}(i)$. The radical part $\sqrt{r_j}$ is normalized to be a product of prime-factors i.e.

$$r_j = 2^{n_0} \times 3^{n_1} \times \dots \times P_j^{n_{j-1}} \times \dots, \quad (2)$$

where P_j is the j -th prime and $n_j \in \{0, 1\}$ for all j . For ease of implementation the radicands are indexed by integers see Sec. 2.2.

The class implements exact arithmetic for addition, subtraction, negation, complex conjugation, multiplication, inversion, division, integer powers, equality, and normalization.

The practical design goal is:

Represent each number as a sparse dictionary from a radical squareclass index to a normalized Gaussian rational coefficient.

The mathematical design goal is:

Work inside finite extensions of $\mathbb{Q}(i)$ generated by square roots: $\mathbb{Q}(i, \sqrt{r_1}, \dots)$ while using only integer arithmetic, sparse dictionaries, and binary vector operations over $\mathbb{F}_2$.

2 Canonical representation

2.1 High-level representation

An element is stored as a dictionary

$$\text{data} : \text{radicand_index} \mapsto [p, q, d]. \quad (3)$$

The entry

$$i \mapsto [p, q, d]$$

represents the term

$$\frac{p + iq}{d} \sqrt{R_i}, \quad (4)$$

where R_i is the squarefree radicand encoded by the integer index i .

The zero number is represented by the empty dictionary:

$$0 \equiv \{\}. \quad (5)$$

A pure Gaussian rational

$$\frac{p + iq}{d}$$

is represented by

$$\{0 : [p, q, d]\}, \quad (6)$$

provided $(p, q) \neq (0, 0)$. The index 0 corresponds to the empty squareclass, i.e. $\sqrt{1} = 1$.

2.2 Encoding squareclasses by bitmasks

Every integer radicand is reduced to its squarefree part. If

$$r = \pm \prod_{k \geq 0} p_k^{n_k}, \quad (7)$$

where $p_0 = 2, p_1 = 3, p_2 = 5, \dots$ are the primes, then only the parities

$$\epsilon_k = n_k \bmod 2 \quad (8)$$

matter for the squareclass.

The squareclass is encoded by the integer bitmask

$$i = \sum_{k \geq 0} \epsilon_k 2^k. \quad (9)$$

Thus:

$$\begin{aligned} \sqrt{1} &\leftrightarrow 0, \\ \sqrt{2} &\leftrightarrow 1, \\ \sqrt{3} &\leftrightarrow 2, \\ \sqrt{5} &\leftrightarrow 4, \\ \sqrt{6} &= \sqrt{2 \cdot 3} \leftrightarrow 1 + 2 = 3, \\ \sqrt{30} &= \sqrt{2 \cdot 3 \cdot 5} \leftrightarrow 1 + 2 + 4 = 7. \end{aligned}$$

The bitmask should be understood as a vector in $\mathbb{F}_2^m$, where each bit records whether a prime appears with odd exponent.

2.3 Multiplication of radical indices

Multiplication of squareclasses is bitwise XOR:

$$\sqrt{R_a} \sqrt{R_b} = \gamma(a, b) \sqrt{R_{a \text{ xor } b}}, \quad (10)$$

where $a \text{ xor } b$ is bitwise XOR. The scalar $\gamma(a, b) \in \mathbb{Z}$ accounts for square factors produced by the product.

If the stored indices already correspond to squarefree radicands R_a, R_b , then

$$R_a R_b = G(a, b)^2 R_{a \text{ xor } b}, \quad (11)$$

so

$$\sqrt{R_a} \sqrt{R_b} = G(a, b) \sqrt{R_{a \text{ xor } b}}. \quad (12)$$

Here $G(a, b)$ is the product of primes whose bits occur in both a and b . Equivalently,

$$G(a, b) = R_{a \& b}, \quad (13)$$

where $a \& b$ is bitwise AND, interpreted as the product of the corresponding primes.

Example:

$$\sqrt{6} \sqrt{10} = \sqrt{60} = 2\sqrt{15}. \quad (14)$$

The indices are

$$6 \leftrightarrow 3 = (011)_2, \quad 10 \leftrightarrow 5 = (101)_2.$$

Then

$$3 \text{ xor } 5 = 6 = (110)_2,$$

corresponding to $15 = 3 \cdot 5$, and

$$3 \& 5 = 1,$$

corresponding to the shared prime 2, so the scalar factor is 2.

3 Normalization invariants

After every public operation, the object should be normalized. The normal form has the following invariants.

N1. The zero element is exactly the empty dictionary $\{\}$.

N2. No stored coefficient is zero. That is, there is no entry $[0, 0, d]$.

N3. Every denominator is positive: $d > 0$.

N4. Each coefficient $[p, q, d]$ is reduced by

$$g = \gcd(p, q, d), \quad (15)$$

replacing

$$[p, q, d] \mapsto [p/g, q/g, d/g]. \quad (16)$$

N5. Like radical indices are combined. There is at most one dictionary entry for each square-class index.

N6. All radicand indices are squarefree bitmask indices. No prime appears more than once in a stored radical basis element.

Equality is then dictionary equality:

$$A = B \iff \text{normalize}(A).\text{data} = \text{normalize}(B).\text{data}. \quad (17)$$

4 Gaussian rational coefficient arithmetic

The coefficient associated to a radical index is stored as

$$[p, q, d] \leftrightarrow \frac{p + iq}{d}. \quad (18)$$

The basic coefficient operations are:

$$\begin{aligned} \frac{p_1 + iq_1}{d_1} + \frac{p_2 + iq_2}{d_2} &= \frac{(p_1 d_2 + p_2 d_1) + i(q_1 d_2 + q_2 d_1)}{d_1 d_2}, \\ -\frac{p + iq}{d} &= \frac{-p - iq}{d}, \\ \overline{\frac{p + iq}{d}} &= \frac{p - iq}{d}, \\ \frac{p_1 + iq_1}{d_1} \frac{p_2 + iq_2}{d_2} &= \frac{(p_1 p_2 - q_1 q_2) + i(p_1 q_2 + q_1 p_2)}{d_1 d_2}. \end{aligned}$$

A nonzero Gaussian rational coefficient is inverted by

$$\left(\frac{p + iq}{d} \right)^{-1} = \frac{d(p - iq)}{p^2 + q^2}. \quad (19)$$

5 Addition, subtraction, negation, and conjugation

5.1 Addition

Addition is dictionary merging.

Given two normalized objects A and B :

- a) Copy one dictionary into the result.
- b) For each term of the other dictionary:
 - i) if the radical index is absent, insert the coefficient;
 - ii) if the radical index is present, add the Gaussian rational coefficients.
- c) Normalize the result, removing zero coefficients.

5.2 Subtraction

Subtraction is implemented as

$$A - B = A + (-B). \quad (20)$$

Negation sends every coefficient

$$[p, q, d] \mapsto [-p, -q, d]. \quad (21)$$

5.3 Complex conjugation

Complex conjugation acts only on i , not on the formal real radicals:

$$\overline{\sqrt{R_i}} = \sqrt{R_i}. \quad (22)$$

Therefore each coefficient changes by

$$[p, q, d] \mapsto [p, -q, d]. \quad (23)$$

6 Multiplication

Multiplication is term-by-term distribution followed by collection.

If

$$A = \sum_a c_a \sqrt{R_a}, \quad B = \sum_b e_b \sqrt{R_b}, \quad (24)$$

then

$$AB = \sum_{a,b} c_a e_b G(a, b) \sqrt{R_{a \text{ xor } b}}. \quad (25)$$

The implementation loops over all pairs of dictionary entries:

```
result = {}
for a, ca in A.data.items():
    for b, cb in B.data.items():
        g = shared_square_factor(a, b)      # product of common primes
        k = a xor b                          # new squareclass index
        coeff = ca * cb * g
        add coeff to result[k]
normalize(result)
```

For sparse objects with M and N terms, raw multiplication takes $O(MN)$ coefficient products before collection.

7 Inversion

7.1 Why inversion is always possible for nonzero elements

Let

$$A = \sum_j c_j \sqrt{R_j}, \quad c_j \in \mathbb{Q}(i). \quad (26)$$

The finitely many radicals appearing in A generate a finite field extension

$$K = \mathbb{Q}(i, \sqrt{R_1}, \dots, \sqrt{R_n}). \quad (27)$$

If $A \neq 0$, then $A^{-1} \in K$. Therefore the inverse can again be represented as a finite sum of Gaussian rational coefficients times square-root basis elements.

The implementation does not need general field machinery. It computes the inverse by repeated rationalization of the denominator.

7.2 Single-term inverse

If

$$A = c\sqrt{R_i}, \quad c \in \mathbb{Q}(i), \quad c \neq 0, \quad (28)$$

then

$$A^{-1} = \frac{1}{cR_i} \sqrt{R_i}. \quad (29)$$

This follows from

$$\frac{1}{c\sqrt{R_i}} = \frac{\sqrt{R_i}}{cR_i}. \quad (30)$$

If $i = 0$, this is just inversion of a Gaussian rational.

7.3 Two-term inverse

If

$$A = x + y, \quad (31)$$

where x and y are two radical terms, then multiply numerator and denominator by

$$x - y. \quad (32)$$

Then

$$A^{-1} = \frac{x - y}{x^2 - y^2}. \quad (33)$$

The denominator has fewer radical components, and in the two-term case it becomes a Gaussian rational after collection.

7.4 General inverse by repeated denominator rationalization

For more than two terms, the inverse is computed by maintaining

$$\frac{N}{D} = \frac{1}{A}, \quad (34)$$

initialized as

$$N = 1, \quad D = A. \quad (35)$$

At every step, choose a squareclass direction and split

$$D = E + \theta O, \quad (36)$$

where θ is a chosen radical generator and neither E nor O contains θ . Then multiply both numerator and denominator by

$$E - \theta O. \quad (37)$$

The new denominator is

$$D' = (E + \theta O)(E - \theta O) = E^2 - \theta^2 O^2. \quad (38)$$

The chosen squareclass direction has been eliminated from the denominator.

Repeat until the denominator is a Gaussian rational. Then invert that final Gaussian rational and multiply it into the numerator.

8 The square-space over $\mathbb{F}_2$

8.1 Support set

The radical indices currently appearing in the denominator form a finite support set

$$S = \text{supp}(D) \subseteq \mathbb{F}_2^m. \quad (39)$$

Each index is a bit vector. The rank of the denominator is

$$\rho = \text{rank}_{\mathbb{F}_2} \langle S \rangle. \quad (40)$$

This rank is the number of independent square-root directions that still need to be eliminated.

The minimum possible number of rationalization steps is ρ , assuming each step eliminates one independent direction.

8.2 Cuts are linear functionals

A rationalization step is determined by a nonzero linear functional

$$\lambda : \mathbb{F}_2^m \rightarrow \mathbb{F}_2. \quad (41)$$

It splits the support into two sets:

$$S_0 = \{v \in S : \lambda(v) = 0\}, \quad (42)$$

$$S_1 = \{v \in S : \lambda(v) = 1\}. \quad (43)$$

This corresponds to writing

$$D = E + \theta O. \quad (44)$$

The conjugating factor is

$$E - \theta O. \quad (45)$$

8.3 Support update after a cut

The new denominator support is contained in

$$S' = (S_0 + S_0) \cup (S_1 + S_1), \quad (46)$$

where addition is XOR in $\mathbb{F}_2^m$. Equivalently,

$$S' = \{u \mathbf{xor} v : u, v \in S_0\} \cup \{u \mathbf{xor} v : u, v \in S_1\}. \quad (47)$$

Actual coefficient arithmetic may cause additional cancellations, so this set is the structural support before accidental zero coefficients are removed.

The exact support can be computed by pairwise XORs inside each side of the cut, while simultaneously computing the coefficients of $E^2 - \theta^2 O^2$.

8.4 Cost of a candidate cut

Let

$$e = |S_0|, \quad o = |S_1|, \quad e + o = |S| = N. \quad (48)$$

Computing the denominator by symmetric squaring takes approximately

$$C(\lambda) = \frac{e(e+1)}{2} + \frac{o(o+1)}{2} \quad (49)$$

term multiplications.

For fixed N , this expression is minimized when e and o are as close as possible. Therefore the first heuristic is:

Prefer cuts whose support is close to half of the current terms.

A stronger one-step heuristic is:

Among balanced cuts, prefer the cut with the smallest resulting support

$$|(S_0 + S_0) \cup (S_1 + S_1)|.$$

9 Greedy inversion strategy

9.1 Precomputation

At the beginning of inversion:

1. Extract the denominator support S .
2. Compute the $\mathbb{F}_2$ -rank ρ .
3. Compute a basis for the active square-space.
4. Compute a corresponding dual basis of linear functionals, or otherwise generate candidate functionals λ .
5. For each candidate, estimate the split size and optionally the exact next support size.

In implementation language, the “orthogonal basis” is best interpreted as a dual system of parity tests. A move is not chosen by a radical term directly; it is chosen by a parity test on the radical bitmasks.

9.2 Candidate ordering

A cheap candidate starts from prime-factor columns. For a prime column k , define

$$B_k = \{v \in S : v_k = 1\}. \quad (50)$$

Its support size is $|B_k|$, and the induced split is

$$|B_k| \quad \text{versus} \quad N - |B_k|. \quad (51)$$

A good single-column candidate has $|B_k|$ close to $N/2$.

More general candidates are XOR combinations of prime columns. If the current candidate support is B and a new prime column is C , then adding that prime to the parity test replaces B by the symmetric difference

$$B \triangle C. \quad (52)$$

The size can be updated by

$$|B \triangle C| = |B| + |C| - 2|B \cap C|. \quad (53)$$

This makes it cheap to greedily improve a candidate by adding columns that move the support closer to half of N .

A practical ordering is:

1. Rank prime columns by closeness of support to $N/2$.
2. Use the best columns as seeds.
3. If a seed has support larger than $N/2$, try adding other columns by XOR to bring the support closer to $N/2$.
4. Also allow adding columns to smaller seeds if it improves balance.
5. For the resulting candidates, compute the actual denominator cost and, if desired, exact next support.

This is a heuristic. It is not a proof of global optimality. It is intended to minimize immediate multiplication cost and reduce expression swell.

9.3 Dynamic programming versus greedy search

The exact minimal number of rationalization steps is the rank ρ . The exact minimum total multiplication cost over all possible orders is a shortest-path or dynamic-programming problem over denominator supports.

A denominator-only dynamic program can be written as:

```
best(S):
    if S == {0}:
        return 0
    if S in memo:
        return memo[S]

    ans = infinity
    for lambda in candidate_functionals(span(S)):
        S0 = {v in S : dot(lambda, v) == 0}
        S1 = {v in S : dot(lambda, v) == 1}

        e = len(S0)
        o = len(S1)
        step_cost = e*(e+1)//2 + o*(o+1)//2

        Snext = xor_pair_support(S0) union xor_pair_support(S1)
        ans = min(ans, step_cost + best(Snext))

    memo[S] = ans
    return ans
```

In practice, the search complexity depends more on ρ than on the number of terms N . There are up to

$$2^\rho - 1 \tag{54}$$

nonzero linear functionals at rank ρ . Therefore the practical policy is:

Regime	Suggested strategy
$N \leq 3$	Direct formulas or hand-simplified rationalization.
$3 < N \leq 10$	Greedy is usually cheaper than setting up full dynamic programming.
$10 < N \leq 50$, small ρ	Dynamic programming or bounded lookahead can be worthwhile.
$10 < N \leq 50$, large ρ	Greedy with exact one-step support calculation, or beam search.
$N > 50$	Greedy heuristics; possibly beam search if expression swell is severe.

10 Special handling for three terms

For three-term denominators, the implementation uses the general rationalization idea but simplifies the cut selection by hand.

The goal is to choose a split that does not separate terms whose radicands share a common factor in a way that immediately recreates unwanted radicals through cross-products. In bitmask language, the cut should avoid an obviously bad split when two radicals have a large overlap in their prime-factor parity vectors.

This three-term rule is a small-case heuristic. It avoids the overhead of building the full square-space search machinery for a denominator so small that the general optimization machinery is unlikely to pay for itself.

11 Division

Division is implemented as multiplication by the inverse:

$$\frac{A}{B} = AB^{-1}, \quad B \neq 0. \quad (55)$$

The divisor is inverted using the inversion logic above, and the result is multiplied by the dividend.

12 Integer powers

The method computes A^n for integer n .

12.1 Negative, zero, and small positive exponents

1. If $n < 0$, first compute A^{-1} , then compute $(A^{-1})^{-n}$.
2. If $n = 0$, return 1, except in the undefined case 0^0 where `DomainError` is raised.
3. If $n = 1$, return A .
4. If $n = 2$ or $n = 3$, use direct multiplication. The exponent is too small for optimization overhead to be useful.

12.2 Single-term powers

If

$$A = c\sqrt{R_i}, \quad (56)$$

then even powers become Gaussian rationals times powers of R_i , and odd powers are that even-power result times A . Explicitly:

$$\begin{aligned} A^{2k} &= c^{2k} R_i^k, \\ A^{2k+1} &= c^{2k+1} R_i^k \sqrt{R_i}. \end{aligned}$$

Because the result has at most one radical term, this case can be exponentiated directly using coefficient arithmetic and integer powers.

12.3 Rank-one square-space powers

If the support of A has rank one over $\mathbb{F}_2$, then A lies in a quadratic extension of $\mathbb{Q}(i)$. Therefore A satisfies a quadratic equation

$$ax^2 + bx + c = 0, \quad (57)$$

with coefficients in $\mathbb{Q}(i)$. Rearranging gives

$$x^2 = -\frac{b}{a}x - \frac{c}{a}. \quad (58)$$

Thus every power of $x = A$ reduces to a linear expression

$$x^n = \alpha_n x + \beta_n. \quad (59)$$

The coefficients $\alpha_n, \beta_n \in \mathbb{Q}(i)$ are computed by matrix exponentiation of the recurrence induced by

$$x^2 = ux + v, \quad (60)$$

where

$$u = -\frac{b}{a}, \quad v = -\frac{c}{a}. \quad (61)$$

Multiplication by x acts on the vector representation $\alpha x + \beta$ by

$$x(\alpha x + \beta) = \alpha x^2 + \beta x = \alpha(ux + v) + \beta x = (\alpha u + \beta)x + \alpha v. \quad (62)$$

Therefore the transition is

$$\begin{pmatrix} \alpha' \\ \beta' \end{pmatrix} = \begin{pmatrix} u & 1 \\ v & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}. \quad (63)$$

Starting from

$$x^1 = 1 \cdot x + 0, \quad (64)$$

one obtains x^n by exponentiating this 2×2 matrix.

12.4 Rank two and higher powers

Rank-two expressions could in principle be optimized by using the degree-four algebra generated by two independent square-root directions. However, the implementation does not currently do this because:

1. the implementation complexity is higher;
2. the practical computational benefit is unclear;
3. multiplication is already exact and simple for moderate exponents.

For general positive exponents outside the optimized cases, the implementation uses repeated multiplication.

13 Equality

Equality is defined by normalized dictionary equality.

Because all terms are collected by squareclass index and all coefficients are reduced, two mathematically equal numbers have the same normalized dictionary representation, assuming all construction paths respect the same squarefree-index convention.

Thus:

$$A == B \quad (65)$$

means:

$$\text{normalize}(A).\text{data} == \text{normalize}(B).\text{data}. \quad (66)$$

14 Implementation-oriented summary

The object represents elements of a sparse $\mathbb{Q}(i)$ -vector space with basis elements $\sqrt{R_i}$, where i is a bitmask encoding the squarefree prime support of R_i .

Concept	Implementation meaning
Gaussian rational coefficient	Triple $[p, q, d]$ representing $(p + iq)/d$.
Radical basis element	Integer bitmask representing squarefree prime support.
Addition	Merge dictionaries and add coefficients with equal keys.
Negation	Negate p and q in every coefficient.
Complex conjugation	Negate only q .
Multiplication	Pairwise term multiplication; new key is XOR of keys; coefficient gets square-factor correction.
Normalization	Reduce coefficient gcds, force positive denominators, combine keys, remove zero terms.
Inversion	Repeatedly rationalize denominator using $D = E + \theta O$, multiply by $E - \theta O$.
Rank	Number of independent square-root directions in the current support.
Cut	Linear parity test $\lambda(v)$ on bitmask vectors.
Greedy cut score	Prefer balanced splits and smaller next support.
Division	Multiply by inverse.
Power	Special cases for negative, zero, small, single-term, and rank-one; otherwise repeated multiplication.
Equality	Normalized dictionary equality.

15 LLM-facing interpretation guide

The code is not trying to implement general symbolic algebra. It implements one specific exact algebra:

$$\mathbb{Q}(i)\text{-linear combinations of square roots of squarefree integers.} \quad (67)$$

The important facts for understanding the code are:

1. The dictionary key is not the original radicand. It is the squarefree squareclass encoded as a prime-parity bitmask.
2. Multiplying radicals means XORing keys, plus multiplying the coefficient by the product of shared primes.
3. Coefficients are exact Gaussian rationals stored as integer triples.
4. Normalization is essential. Without normalization, equality by dictionary comparison is invalid.
5. Inversion works because finitely many square roots generate a finite field extension, and every nonzero field element has an inverse in that same field.
6. The inversion algorithm is repeated rationalization, not approximate numerical inversion.
7. The $\text{GF}(2)$ rank measures how many independent radical directions remain in a denominator.
8. A rationalization step is a parity cut of the current radical support.
9. Greedy inversion is an optimization heuristic for reducing multiplication cost and expression swell. It is not guaranteed globally optimal.

10. Rank-one powers are optimized because the element satisfies a quadratic equation, so all powers reduce to $\alpha A + \beta$.

These points should be enough to understand why the algorithms are correct and why the implementation uses bitmasks, XOR, dictionary collection, and repeated conjugation.

16 Potential edge cases

The implementation should explicitly handle or reject the following cases:

1. Inversion of zero is undefined $\rightarrow$ `MathDomainError`.
2. Division by zero is undefined $\rightarrow$ `MathDomainError`.
3. 0^0 is undefined $\rightarrow$ `MathDomainError`.
4. Denominators in coefficient triples must never be zero. Zero is represented as $0 = \{\}$ and zero terms from a non-zero `QINumber` are removed from its dictionary.
5. Denominators should be normalized to positive values.
6. Zero coefficients must be removed from the dictionary.
7. Original radicands must be reduced to squarefree squareclass indices before storage.
8. Negative radicands are not allowed. Attempts to pass negative radicands $\rightarrow$ `MathDomainError` with the justification that the sign of negative radicands is arbitrary, as $\sqrt{-1} = \pm i$.